Spec 001 — Delete job from tablet

Status: Draft — awaiting review **Authored**: 2026-04-22 **Touches**: `lib/domain/ports/git_port.dart`, `lib/infra/git/`, `lib/app/controllers/`, `lib/ui/screens/job_list/job_list_screen.dart` **Capacitor**: no regressions — tablet-only feature

---

1. Problem

The tablet app has no way to remove a pending job. Jobs accumulate under `jobs/pending/<jobId>/` on the `claude-jobs` branch and can only transition forward: spec → annotate → submit review → approve. Once a job lands on-device there's no "abandon", "restart", or "wipe" action; the only way to clear one today is to manually edit the working tree outside the app or to re-clone the repo.

The immediate need is "start fresh": during testing the user wants to re-annotate the same spec from scratch, which today means hand-deleting files from a desktop shell. We need a first-class tablet action that removes a job and lands the removal as a commit on `claude-jobs` so other devices (and the remote) stay in sync.

2. Proposed Change

Add a **Delete job** action, accessed by **long-pressing a job row** in the JobList. The action:

1. Shows a confirmation bottom sheet with the jobId + a destructive-colored "Delete" button.
2. On confirm, removes the entire `jobs/pending/<jobId>/` folder from the working tree *and* the git index, then creates a commit on the `claude-jobs` branch authored by the signed-in identity with message `delete: <jobId>`.
3. Clears the matching review-draft autosave file under `appDocs/drafts/<jobId>/`.
4. Refreshes the JobList so the row disappears. The user's next **Sync Up** pushes the deletion to the remote.

"Start fresh" = the user re-imports the spec from its source repo via the existing repo-browser flow.

2a. Scope — what changes

- `lib/domain/ports/git_port.dart` — extend `GitPort.commit()` with an optional `removals: List<String>` parameter (paths relative to workdir). Removals + writes are applied to the same commit; empty removals preserves current behavior. `FileWrite` stays unchanged (it's write-only); removals are a parallel list to avoid overloading that type.
- `lib/infra/git/_git_messages.dart` — `GitReqCommit.removals` field added, serialized through the isolate channel.
- `lib/infra/git/_git_isolate.dart` `_handleCommit` — for each path in `removals`, `File('$workdir/$path').deleteSync()` and `index.remove(path)`, before `index.writeTree()`. Missing files are tolerated (no-op) to keep the op idempotent.
- `lib/infra/git/fake_git_port.dart` — fake records `removals` in the commit log + strips matching paths from the in-memory branch snapshot, mirroring the real adapter.
- **New** `lib/app/controllers/job_deleter.dart` — pure domain-style service (no Flutter):

  ```
  class JobDeleter {
    JobDeleter({
      required FileSystemPort fs,
      required GitPort git,
      required ReviewDraftStore drafts,
    });
    Future<Commit> delete({
      required JobRef job,
      required String workdir,
      required GitIdentity id,
    });
  }
  ```

  Enumerates files under `$workdir/jobs/pending/<jobId>/` via `FileSystemPort.listDir` (recursive walk), collects their repo-relative paths, calls `git.commit(files: [], removals: paths, message: 'delete: <jobId>', branch: 'claude-jobs', id: id)`, and `drafts.delete(job)`. Returns the commit.
- **New** `lib/app/providers/job_deleter_providers.dart` — `jobDeleterProvider` wires the service; depends on `fileSystemProvider`, `gitPortProvider`, `reviewDraftStoreProvider`.
- `lib/ui/screens/job_list/job_list_screen.dart` — `_JobRow` gets an `onLongPress` on its `InkWell` that shows a `showModalBottomSheet`. The sheet has:
  - Title: "Delete job"
  - Subtitle: monospace jobId
  - Body: "Removes `02-spec.md`, review, and annotation files. Commits to `claude-jobs`. Next Sync Up pushes the deletion."
  - Two actions: **Cancel** (ghost) and **Delete** (red elevated).
  - On Delete: dismiss sheet, show a SnackBar "Deleting...", await `jobDeleter.delete(...)`, on success invalidate `jobListControllerProvider` + toast "Deleted", on failure toast the error. Identity comes from the existing auth session; if signed out, the sheet's Delete button is disabled with a hint.

2b. Scope — what stays the same

- Sync / Approve / Submit / Annotate flows — no changes.
- Commit-planner invariants — not involved; the delete commit doesn't go through `CommitPlanner`.
- Legacy SVG + PNG annotation writes — still emitted alongside PDF + JSON per the prior decision.
- Existing tests for `GitPort.commit()` keep passing (empty `removals` default matches the old contract).
- No new "Undo delete" action; the commit is on the branch and recoverable via standard git if needed.

3. Implementation notes

- **Idempotency.** `File.deleteSync` tolerates missing; `index.remove` no-ops on paths not in the index. Running Delete twice produces one commit then a no-op (empty tree change → we detect and skip the commit rather than writing an empty one).
- **Empty commit guard.** Before creating the commit in the isolate, compare `index.writeTree()` against `HEAD^{tree}`; if identical, return a typed `CommitNoop` outcome that `JobDeleter` maps to "nothing to delete" instead of throwing. Keeps the UI honest when the user long-presses a phantom row.
- **Order matters.** Clear the draft *before* committing — a crash between them leaves a stale draft pointing at a gone job, but no lost work (the job is the source of truth).
- **Identity.** Same `GitIdentity` the Submit Review flow uses (read from `authPortProvider.currentSession`). If the user is signed out, the action is disabled at the sheet level so we never surface a half-baked commit prompt.

4. UI — where it lives

- JobList sidebar only. Annotate screen does NOT gain a delete affordance — too easy to fat-finger mid-review.
- Long-press is the only trigger. No visible trash icon on the row (keeps the row clean; matches the mobile destructive-action pattern).
- Sheet uses the existing token palette (`t.statusDanger` for the destructive button).

5. Test cases

5a. Domain / unit

- `test/app/controllers/job_deleter_test.dart` (new):
  - Happy path: seeded FakeGitPort + FakeFileSystem with a pending job folder. `delete()` produces a commit whose `removals` list matches the files under the folder; the fake's branch snapshot no longer contains those paths. Draft at `appDocs/drafts/<jobId>/03-review.md.draft` is gone.
  - Empty folder: if `jobs/pending/<jobId>/` is empty (race with another client), `delete()` returns `CommitNoop` and doesn't touch git.
  - Signed-out caller: not reachable here (UI disables it); unit test asserts the service requires a non-null identity by type.
- `test/domain/ports/git_port_test.dart` (or equivalent): round-trip a `commit(removals: [...])` through `FakeGitPort` and assert the paths are excluded from the resulting snapshot.
- `test/infra/git/_git_isolate_test.dart` (if it exists for isolate-level tests): assert `_handleCommit` removes files from disk + index for each `removals` entry.

5b. UI / widget

- `test/ui/screens/job_list/job_list_delete_test.dart` (new):
  - Long-press a `_JobRow` → bottom sheet appears with the correct jobId.
  - Tap Cancel → sheet dismisses, no git call.
  - Tap Delete with a seeded signed-in identity → `FakeGitPort.commits` records a commit on `claude-jobs` with `message == 'delete: <jobId>'` and `removals` non-empty; job disappears from the next render.
  - Signed-out state → Delete button is disabled.

5c. Manual

- OnePlus Pad Go 2 / landscape. Open a repo with ≥1 pending job. Long-press the row. Confirm. Row disappears. Sync Up. Verify on GitHub that the `claude-jobs` branch has a new `delete: <jobId>` commit and the folder is gone on remote.

6. Open Questions

- **Feedback on destructive action** — SnackBar is lightweight; worth a full banner with Undo? Current plan: no Undo, SnackBar is enough (git log is the real audit trail). Enough
- **Long-press threshold** — use platform default (500 ms)? Seems fine; revisit if users hit it accidentally. OK
- **Delete while mid-review** — if the user has a draft open in review panel and we delete the job from the list, should we pop any open screen pointing at the gone job? Current plan: yes — `JobListController.invalidate` + a `Navigator.popUntil` on the root. Easy to miss; flag during QA. YES DELETE

7. Critical files (reference)

- `lib/domain/ports/git_port.dart` — `GitPort.commit()` signature change
- `lib/infra/git/_git_messages.dart`, `_git_isolate.dart`, `git_adapter.dart` — isolate wiring
- `lib/domain/fakes/fake_git_port.dart` — keep fake in sync with the real port
- `lib/app/controllers/job_deleter.dart` (new) — domain service
- `lib/app/controllers/review_draft_store.dart:61-64` — reuses existing `delete(JobRef)`
- `lib/app/controllers/job_list_controller.dart:73-76` — reuse `refresh()` for post-delete reload
- `lib/ui/screens/job_list/job_list_screen.dart:822-912` — `_JobRow` long-press + sheet

8. Verification plan

1. `flutter analyze` clean.
2. Unit tests green (including the new `job_deleter_test.dart`). looks good
3. Manual run on the tablet: long-press → confirm → row gone → Sync Up → branch diff on GitHub shows the folder removed. Approved
4. Regression: re-run the existing Submit Review happy-path test; `commit(removals: [])` default must match the old behavior byte-for-byte.